

Debug Options Watch Guide

What every Logging setting observes, where it lives in the code, and when to turn it up.

How to read the controls

Each picker is a threshold for one stable logging category. Selecting a level includes that level and everything more severe. The option does not enable a separate subsystem or change show behavior; it changes which existing diagnostic events are accepted by Prism's logging pipeline.

Setting	You will receive	Operational meaning
Off	Error + Fault	Routine activity is hidden, but failures are never silenced.
Errors only	Error + Fault	Same practical threshold as Off; emphasizes failure-only use.
High-level	Notice and above	Lifecycle and important state transitions; the production norm.
Information	Info and above	Successful operations and useful state changes.
Verbose	Debug and above	Fine-grained decisions and sampled summaries for active troubleshooting.

Example: App > Lifecycle = Verbose

This watches Prism's application bootstrap and termination code in AppModel. You will see launch and quit milestones such as `app.lifecycle.launch` and `app.lifecycle.terminate`. It does not watch cue playback, DMX output, window layout, or AME; those have their own categories.

Important volume and privacy behavior

Verbose AME, scheduler, and DMX paths use sampling or rate limiting instead of logging each event or frame. Dynamic technical values are private in macOS Unified Logging. Error and fault events remain enabled even when a category is Off.

App

Application shell, preferences, and window lifecycle.

Lifecycle `app.lifecycle`

WATCHES	Prism launch milestones, successful startup, shutdown, and termination.
CODE AREA	AppModel and application bootstrap/termination paths
EVENTS	<code>app.lifecycle.launch</code> ; <code>app.lifecycle.terminate</code>
ENABLE WHEN	Use Verbose when Prism hangs during launch or quit, or when you need to reconstruct startup order.

Settings `app.settings`

WATCHES	Loading, applying, persisting, and falling back from application-wide logging preferences.
CODE AREA	AppSettingsStore and logging bootstrap
EVENTS	<code>app.settings.profile_changed</code> ; <code>app.settings.config_fallback</code>
ENABLE WHEN	Use Information/Verbose when a selected profile or category level does not persist or apply as expected.

Windowing `app.windowing`

WATCHES	Creation and opening of Prism floating workspace surfaces and auxiliary windows.
CODE AREA	AppModel window/surface coordination
EVENTS	<code>app.windowing.surface_opened</code>
ENABLE WHEN	Use Information when a floating surface fails to appear or opens unexpectedly.

Project

Show document storage, autosave, schema migration, and validation.

Document `project.document`

WATCHES	Creating, opening, saving, importing, and command rollback/failure activity for a show package.
CODE AREA	ProjectPackage, DocumentSession, ProjectController, AppModel
EVENTS	<code>project.document.created/opened/saved</code> ; <code>project.command.failed/rolled_back</code>
ENABLE WHEN	Use Verbose for save/open problems, failed undo/redo, damaged packages, or document command failures.

Autosave `project.autosave`

WATCHES	Autosave attempts, success, failure counts, and automatic disabling after repeated problems.
CODE AREA	ProjectController autosave coordination
EVENTS	<code>project.autosave.succeeded/failed/disabled</code>
ENABLE WHEN	Use Information while diagnosing missing autosaves; errors remain logged even when Off.

Migration `project.migration`

WATCHES	Schema upgrades performed while opening older Prism/Aurora show packages.
CODE AREA	SchemaMigration and ProjectPackage load
EVENTS	<code>project.document.migrated</code>
ENABLE WHEN	Use High-level/Information when an older show changes during open or fails after an app upgrade.

Validation `project.validation`

WATCHES	Aggregated project integrity and resolution diagnostics produced for the Diagnostics panel.
CODE AREA	DiagnosticsController and project validators
EVENTS	<code>project.validation.summary</code>
ENABLE WHEN	Use Information when fixtures, references, cue data, or project consistency checks report issues.

Fixture

Fixture personality catalogs and import pipelines.

Library `fixture.library`

WATCHES	Built-in/user fixture library loading, folder changes, library import/export, and save failures.
CODE AREA	FixtureLibrary, ProjectController, fixture-library UI
EVENTS	<code>fixture.library.loaded/load_failed/directory_changed/import_failed/export_failed</code>
ENABLE WHEN	Use Information when personalities are missing, duplicated, or the library directory will not reload.

Import `fixture.import`

WATCHES	Generic fixture-definition import completion and failures.
CODE AREA	FixtureImporter and ProjectController import boundary
EVENTS	<code>fixture.import.completed; fixture.import.failed</code>
ENABLE WHEN	Use Information/Verbose when a third-party fixture definition is rejected or imports incorrectly.

LightKey `fixture.lightkey`

WATCHES	LightKey fixture parsing, conversion, completion, and import failures.
CODE AREA	LightKeyFixtureImporter and its import window
EVENTS	<code>fixture.lightkey.import_completed/import_failed</code>
ENABLE WHEN	Use Verbose for LightKey-specific conversion or compatibility problems.

Show Engine

Live show execution, programmer state, cues, effects, and frame health.

Show engine `engine.show`

WATCHES	Starting/stopping the lighting engine and failures that prevent show execution.
CODE AREA	ShowControlController and LightingEngine startup
EVENTS	<code>engine.show.started/start_failed/stopped</code>
ENABLE WHEN	Use High-level for lifecycle history; Verbose when the show engine will not start.

Programmer `engine.programmer`

WATCHES	Programmer state operations, currently centered on explicit clear actions.
CODE AREA	AuroraEngine Programmer
EVENTS	<code>engine.programmer.clear</code>
ENABLE WHEN	Use Information to confirm programmer clears and trace unexpected loss of programmer values.

Cues `engine.cues`

WATCHES	GO actions and cue playback transitions owned by the lighting engine.
CODE AREA	LightingEngine cue execution
EVENTS	<code>engine.cues.go</code>
ENABLE WHEN	Use High-level to reconstruct operator GO activity during playback.

Effects `engine.effects`

WATCHES	Failures while applying or updating effect definitions and effect-related commands.
CODE AREA	Effects UI/commands and error-reporting boundary
EVENTS	<code>engine.effects.update_failed</code>
ENABLE WHEN	Use Verbose when effect edits fail; successful per-frame effect evaluation is intentionally not logged.

Performance `engine.performance`

WATCHES	Frame-time budget overruns and engine performance health, rate-limited to avoid log storms.
CODE AREA	EngineFrameMetrics and performance budget checks
EVENTS	<code>engine.performance.budget_exceeded</code>
ENABLE WHEN	Use Verbose during stutter/latency diagnosis. This watches timing summaries, not every rendered frame.

AME

Automation Mapping Engine input-to-action pipeline.

Ingress `ame.ingress`

WATCHES	Incoming normalized events, arm/disarm, configuration acceptance/rejection, timestamp fallback, and learn failures.
CODE AREA	AMERuntime input boundary
EVENTS	<code>ame.ingress.received/armed/disarmed/config_accepted/invalid_config</code>
ENABLE WHEN	Use Verbose to prove that an input reached AME before matching begins.

Matching `ame.matching`

WATCHES	Trigger and mapping candidates, matches, misses, disabled/suppressed mappings, scope/timing rejection, and rate suppression.
CODE AREA	AMERuntime mapping selection
EVENTS	<code>ame.matching.matched/missed/candidate/suppressed/scope_inactive</code>
ENABLE WHEN	Use Verbose when AME receives input but the expected mapping does not fire.

Transform `ame.transform`

WATCHES	Input-value transformation rejection before an action can be emitted.
CODE AREA	AME transform/evaluation stage
EVENTS	<code>ame.transform.rejected</code>
ENABLE WHEN	Use Verbose for scale, range, or value-conversion problems.

Sequence `ame.sequence`

WATCHES	Sequence lookup, deferred steps, step firing, advancement, reset, empty/missing/invalid steps, and control actions.
CODE AREA	AMESequenceRuntime and AMERuntime
EVENTS	<code>ame.sequence.step_fired/advanced/reset/deferred/missing/invalid_step</code>
ENABLE WHEN	Use Verbose when a sequence fires the wrong step, stalls, or has no valid context.

Quantization `ame.quantization`

WATCHES	Immediate/deferred scheduling decisions, cancellations, held actions, and fallback behavior at musical boundaries.
CODE AREA	AME quantization bridge and runtime
EVENTS	<code>ame.quantization.immediate/deferred/cancelled/held/fallback</code>
ENABLE WHEN	Use Verbose when an AME action is early, late, cancelled, or never released on beat/bar boundaries.

AME (continued)

Final action emission and held-state ownership.

Emission `ame.emission`

WATCHES	Final action decision: fired, skipped, dry-run, armed, or unsupported by the live executor.
CODE AREA	AMERuntime output gate into Prism actions
EVENTS	<code>ame.emission.fired/skipped/dry_run/armed/unsupported</code>
ENABLE WHEN	Use Information for actual fires; Verbose to explain why a matched mapping did not execute.

Held state `ame.heldState`

WATCHES	Acquisition and release of held/toggle state, including cleanup on context, document, mode, or source changes.
CODE AREA	AMERuntime held-state ownership
EVENTS	<code>ame.held.acquired/released/release_all/released_by_source/sim_purged</code>
ENABLE WHEN	Use Verbose for stuck toggles, missing note-off behavior, or state that survives a context change.

Music Engine

Musical timing, transport, scheduling, and song section state.

Clock music.clock

WATCHES	Clock-provider changes/loss and changes to the active tempo source.
CODE AREA	MusicalEngine timing-provider selection
EVENTS	music.clock.provider_changed/provider_lost/tempo_source_changed
ENABLE WHEN	Use High-level/Information when tempo authority jumps, disappears, or selects the wrong provider.

Transport music.transport

WATCHES	Start, continue, and stop transitions of musical transport.
CODE AREA	MusicalEngine transport
EVENTS	music.transport.start/continue/stop
ENABLE WHEN	Use High-level to reconstruct transport operation independently from AME.

Scheduler music.scheduler

WATCHES	Scheduler startup/rescheduling, missed deadlines, and rate-limited tick summaries.
CODE AREA	MusicalEngineRuntimeDriver and scheduler
EVENTS	music.scheduler.rescheduled/deadline_missed/tick_summary
ENABLE WHEN	Use Verbose for timing jitter or missed musical deadlines; tick output is summarized at about 1 Hz.

Song music.song

WATCHES	Active song-section changes used by song-aware playback and AME scope.
CODE AREA	SongDirector
EVENTS	music.song.section_changed
ENABLE WHEN	Use Information when Prism enters the wrong section or section-dependent mappings behave unexpectedly.

Output

Routing and physical/network DMX transmission.

Routing output.routing

WATCHES	Changes to active output drivers and universe-to-driver routing.
CODE AREA	OutputManager
EVENTS	output.routing.changed
ENABLE WHEN	Use High-level when universes are sent to the wrong driver or an output route does not activate.

Local DMX output.localDMX

WATCHES	ENTTEC USB DMX Pro startup/shutdown, serial writes, failures/recovery, and sampled frame counts.
CODE AREA	ENTTECUSBDMXProDriver
EVENTS	output.localDMX.started/stopped/failed/recovered/frame_summary
ENABLE WHEN	Use Verbose to verify sustained USB frame delivery; frame summaries are capped near 1 Hz.

Art-Net output.artnet

WATCHES	Art-Net connection lifecycle, asynchronous send failures/recovery, and sampled frame counts.
CODE AREA	ArtNetOutputDriver
EVENTS	output.artnet.started/stopped/failed/recovered/frame_summary
ENABLE WHEN	Use Verbose for network-output dropouts or to confirm Art-Net frames continue flowing.

sACN output.sacn

WATCHES	sACN lifecycle, asynchronous send failures/recovery, multicast/unicast delivery, and sampled frame counts.
CODE AREA	SACNOutputDriver
EVENTS	output.sacn.started/stopped/failed/recovered/frame_summary
ENABLE WHEN	Use Verbose for E1.31 delivery failures or to verify sustained output.

Control

MIDI, OSC, RTP-MIDI, and keyboard control inputs.

MIDI `control.midi`

WATCHES	CoreMIDI startup, source connection/removal, connection failures, learned mappings, and output activity summaries.
CODE AREA	MIDIInputManager, MIDIOutputManager, InputController
EVENTS	<code>control.midi.started/failed/source_changed/learned</code>
ENABLE WHEN	Use Information when a controller disappears, reconnects, or MIDI Learn chooses the wrong input.

OSC `control.osc`

WATCHES	OSC listener lifecycle/failures and verbose action dispatch confirmation.
CODE AREA	OSCInputServer and AppModel action dispatch
EVENTS	<code>control.osc.started/stopped/failed/dispatch</code>
ENABLE WHEN	Use Verbose when packets arrive at the Mac but do not appear to trigger a Prism action.

RTP-MIDI `control.rtpMIDI`

WATCHES	Network MIDI session start/stop and related controller lifecycle.
CODE AREA	InputController RTP-MIDI coordination
EVENTS	<code>control.rtpMIDI.started/stopped</code>
ENABLE WHEN	Use High-level when a network MIDI session will not advertise or remain active.

Keyboard `control.keyboard`

WATCHES	Reserved category for keyboard command routing and shortcut diagnostics.
CODE AREA	Keyboard command gate/routing (category currently has no dedicated production event)
EVENTS	No dedicated event codes in this pass
ENABLE WHEN	Changing this option currently has little visible effect; use UI/Workspace for focus and window-related shortcut problems.

Remote

Remote TCP/web hosting, authentication, and protocol decoding.

Host remote.host

WATCHES	Overall Remote Access lifecycle, listener state, command receipt, and host startup failures.
CODE AREA	RemoteHost, RemoteController, AppModel remote command bridge
EVENTS	remote.host.starting/started/stopped/failed
ENABLE WHEN	Use Verbose when Remote Access starts inconsistently or commands reach the host but do not execute.

Web remote.web

WATCHES	HTTP/web listener startup, shutdown, and bind/runtime failures.
CODE AREA	RemoteWebServer
EVENTS	remote.web.started/stopped/failed
ENABLE WHEN	Use High-level/Verbose when the browser client cannot reach Prism's remote web endpoint.

Session remote.session

WATCHES	Client authentication success/failure, session admission, and operator-initiated kicks.
CODE AREA	RemoteSessionManager and RemoteController
EVENTS	remote.session.authenticated/auth_failed/kicked
ENABLE WHEN	Use Information for sign-in history; Warning captures failed authentication without exposing PINs or names.

Codec remote.codec

WATCHES	Malformed or unsupported remote protocol messages during encode/decode.
CODE AREA	RemoteMessages / RemoteCodec
EVENTS	remote.codec.invalid
ENABLE WHEN	Use Verbose when a client connects but requests are rejected as malformed or version-incompatible.

UI / Stage / Workspace

User-interface command failures, layout persistence, and presentation tooling.

Stage ui.stage

WATCHES	Stage layout commits, stage media loading, and stage-edit command failures.
CODE AREA	StageCanvasView, StagePanel, StageStockCatalog
EVENTS	ui.stage.layout_commit_failed; ui.stage.media_failed
ENABLE WHEN	Use Verbose while diagnosing failed stage edits or media assets that will not load.

Patch ui.patch

WATCHES	Patch workspace command failures, fixture patching, and rename failures.
CODE AREA	PatchPanel, PatchWorkspaceView, FixtureBrowserPanel
EVENTS	ui.patch.command_failed; patch.rename.failed
ENABLE WHEN	Use Verbose when UI patch operations fail even though project validation looks healthy.

Workspace ui.workspace

WATCHES	Workspace layout load/save/reset and floating-window frame persistence failures.
CODE AREA	WorkspaceLayoutStore and FloatSurfaceID
EVENTS	ui.workspace.layout_load_failed/layout_save_failed/layout_reset/float_persist_failed
ENABLE WHEN	Use Verbose for panels that reopen in the wrong place or layouts that do not persist.

Presentation ui.presentation

WATCHES	Screenshot/export failures from Prism's presentation and checkpoint tooling.
CODE AREA	Checkpoint screenshot exporters
EVENTS	ui.presentation.export_failed
ENABLE WHEN	Use Verbose when a presentation image or diagnostic screenshot export silently fails.

Troubleshooting shortcuts

Prism will not finish launching	App > Lifecycle; App > Settings; Show Engine > Show engine
A MIDI/OSC action reaches Prism but does not fire	Control > MIDI or OSC; AME > Ingress; AME > Matching; AME > Emission
An AME action fires at the wrong musical time	AME > Quantization; Music Engine > Clock; Music Engine > Scheduler
Lights freeze or network DMX drops	Output > Routing; then Local DMX, Art-Net, or sACN; Show Engine > Performance
A show will not open or save	Project > Document; Project > Migration; Project > Validation; Project > Autosave
A remote client cannot connect	Remote > Host; Remote > Web; Remote > Session; Remote > Codec
Panels or floating windows reopen incorrectly	App > Windowing; UI / Stage / Workspace > Workspace

Source of truth

This guide maps the Settings labels defined by `PrismLogCategory` to the implemented structured events in Prism's source and event catalog. Category keys are stable preference and Unified Logging identifiers.

Subsystem: `com.aurora.lighting`